



SRI SATHYA SAI INSTITUTE OF HIGHER LEARNING
(Deemed to be University)
Department of Mathematics and Computer Science

LAB EXAM 1 - SOLUTIONS

On Newton–Raphson, Gradient Descent/Ascent, linprog, Univariate Minimisation, minimize

UMAM 502: Optimization Techniques for Machine Learning	Instructor: Dr. D Bhanu Prakash	Date: 07-08-2026
--	--	------------------

Module 1: Newton–Raphson

Q1 Find the mistake

This function should minimize f by Newton–Raphson, but it diverges. For $f'(x) = 2x - 4$ and $f''(x) = 2$, starting at $x_0 = 0$, it returns -14.0 after 3 iterations instead of 2.0 .

```
1 def newton(df, d2f, x0, tol=1e-8, maxit=50):
2     x = x0
3     for k in range(maxit):
4         step = df(x) / d2f(x)
5         x = x + step
6         if abs(step) < tol:
7             break
8     return x
```

[A] The buggy line number is _____

[B] Replace it with: _____

Answer: [A] 5 [B] $x - \text{step}$

Why: The Newton update is $x - f/f'$. Adding the step moves uphill along the root-finding direction: $0 \rightarrow 4 \rightarrow -4 \rightarrow 12 \dots$ oscillates and grows.

Q2 Complete the code

Complete this n-dimensional Newton minimiser. It should never form the inverse of the Hessian.

```
import numpy as np

def newton_nd(grad, hess, x0, tol=1e-8, maxit=50):
    x = np.array(x0, dtype=float)
    for k in range(maxit):
        g = grad(x)
        if np.linalg.norm(g) < tol:
            break
        p = np.linalg.____A____(hess(x), g)
        x = x ____B____ p
    return x, k
```

[A] Function name: np.linalg._____

[B] Operator: x ____ p (+ or -)

Answer: [A] solve [B] -

Why: p solves $H p = -\nabla f$, so the step is $x - p$. solve is faster and more accurate than $\text{np.linalg.inv}(H) @ g$. For $f = 3x^2 + 2xy + 2y^2 - 4x - 6y$ it reaches (0.2, 1.4) in one step.

Module 2: Gradient Descent / Ascent

Q3 Predict the output

$f(x, y) = 3x^2 + y^2$, so the gradient is $(6x, 2y)$. What does this code print?

```
import numpy as np

grad = lambda v: np.array([6*v[0], 2*v[1]])
v = np.array([1.0, 4.0])
for _ in range(1):
    v = v - 0.1 * grad(v)
print(v)
```

[A] The first printed value $v[0] =$ _____

[B] The second printed value $v[1] =$ _____

Answer: [A] 0.4 [B] 3.2

Why: Each step multiplies x by $1 - 0.1 \cdot 6 = 0.4$ and y by $1 - 0.1 \cdot 2 = 0.8$. After 1 step: $0.4 = 0.4$ and $4 \cdot 0.8 = 3.2$.

Q4 Find the mistake

This gradient ASCENT routine should maximise $f(x) = -(x - 3)^2$, with $df(x) = -2(x - 3)$. It is called as `grad_ascent(df, 0.0)`.

```
1 def grad_ascent(df, x0, alpha=0.1, maxit=3):
2     x = x0
3     g = df(x)
4     for k in range(maxit):
5         x = x + alpha * g
6     return x
```

[A] The line that must be moved inside the loop is line _____

[B] The buggy code returns _____ (the correct code returns 1.464)

Answer: [A] 3 [B] 1.8 (tol 0.01)

Why: The gradient is computed once and never refreshed. g stays 6, so $x = 0 + 3 \cdot (0.1 \cdot 6) = 1.8$. The fix is to put $g = df(x)$ inside the for loop, giving $0.6 \rightarrow 1.08 \rightarrow 1.464$.

Module 3: `scipy.optimize.linprog`

Q5 Find the mistake

The goal is to MAXIMISE profit $40x_1 + 30x_2$ subject to $x_1 + x_2 \leq 12$, $2x_1 + x_2 \leq 16$, $x \geq 0$. The programmer reports an unexpected result.

```
from scipy.optimize import linprog

c = [40, 30]
A_ub = [[1, 1], [2, 1]]
b_ub = [12, 16]
res = linprog(c, A_ub=A_ub, b_ub=b_ub)
print("Max profit =", res.fun)
```

The code as written prints Max profit = _____

Answer: 0.0 **Why:** `linprog` always minimises. With positive costs the minimum is at $x = (0, 0)$, giving 0.

Q6 Complete the code

Minimise $2x_1 + x_2$ subject to $x_1 + x_2 \geq -1$, where $x_1 \geq 2$ and x_2 is a FREE variable (it may be negative). Complete the bounds argument.

```
from scipy.optimize import linprog

res = linprog(c=[2, 1],
              A_ub=[[-1, -1]], b_ub=[1],
              bounds=[____A____, ____B____])
print(res.fun, res.x)    # expected: 1.0 [ 2. -3.]
```

[A] Bounds tuple for x_1 : _____

[B] Bounds tuple for x_2 : _____

Answer: [A] (2, None) [B] (None, None)

Why: None means “no bound” on that side. Without bounds, SciPy assumes $x \geq 0$ for every variable and returns 0.0 at (0, 0), which is wrong because x_2 should be allowed to reach -3 .

Module 4: Univariate Minimisation

Q7 Predict the output

Notice where the unconstrained minimum of $f(x) = x + 4/x$ lies ($x = 2$) relative to the bounds.

```
from scipy.optimize import minimize_scalar

f = lambda x: x + 4/x
res = minimize_scalar(f, bounds=(3, 10), method='bounded')
print(round(res.x, 2), round(res.fun, 2))
```

[A] $\text{round}(\text{res.x}, 2) =$ _____

[B] $\text{round}(\text{res.fun}, 2) =$ _____

Answer: [A] 3.0 [B] 4.33 (tol 0.01)

Why: $x = 2$ is outside $[3, 10]$. $f' = 1 - 4/x^2 > 0$ on the whole interval, so f is increasing and the minimum sits at the left boundary: $f(3) = 3 + 4/3 = 4.33$.

Q8 Find the mistake

This call crashes before returning a result.

```
import numpy as np
from scipy.optimize import minimize_scalar

f = lambda x: x**4 - 3*x**2 + x + 5
res = minimize_scalar(f, bracket=(-3, -2, 0))
print(res.x)
```

[A] The exception type raised is _____

[B] With the valid bracket $(-3, -1.5, 0)$, $\text{round}(\text{res.x}, 2) =$ _____

Answer: [A] ValueError [B] -1.30 (tol 0.01)

Why: A bracket needs $f(b) < f(a)$ and $f(b) < f(c)$. Here $f(-3) = 56$, $f(-2) = 7$, $f(0) = 5$, so $f(-2)$ is not below $f(0)$. With $b = -1.5$, $f = 1.81$ is below both ends, and Brent converges to $x \approx -1.3008$.

Module 5: Constrained & Unconstrained minimize

Q9 Find the mistake

The aim is to minimise $x^2 + y^2$ subject to $x + y \geq 2$, but the answer is clearly wrong.

```
from scipy.optimize import minimize

f = lambda v: v[0]**2 + v[1]**2
cons = {'type': 'ineq', 'fun': lambda v: 2 - v[0] - v[1]}
res = minimize(f, [3, 3], method='SLSQP', constraints=cons)
print(res.fun)
```

[A] The buggy code prints $\text{res.fun} \approx$ _____

[B] The correct lambda body is: $\text{lambda } v:$ _____

Answer: [A] 0.0 (tol 0.01) [B] $v[0] + v[1] - 2$

Why: 'ineq' means $\text{fun}(x) \geq 0$. The buggy lambda enforces $2 - x - y \geq 0$, i.e. $x + y \leq 2$, which $(0, 0)$ satisfies. The correct body gives the optimum $(1, 1)$ with $f = 2$.

Q10 Complete the code

Minimise $(x - 4)^2 + (y - 2)^2$ subject to $x + y \leq 4$ using LinearConstraint.

```
import numpy as np
from scipy.optimize import minimize, LinearConstraint

f = lambda v: (v[0] - 4)**2 + (v[1] - 2)**2
lin = LinearConstraint([[1, 1]], lb=____A____, ub=4)
res = minimize(f, [0, 0], constraints=lin)
print(res.fun)
```

[A] The value for lb is _____

[B] The printed res.fun $\approx$ _____

Answer: [A] -np.inf [B] 2.0 (tol 0.01)

Why: LinearConstraint encodes $lb \leq Ax \leq ub$, so a one-sided “ $\leq$ ” needs $lb = -\infty$. The unconstrained optimum (4, 2) has $x + y = 6$, which is infeasible. The constrained optimum is (3, 1), where $f = 1 + 1 = 2$.

Answer key (summary)

Q	Blank A	Blank B	Type
Q1	5	x - step	Debug
Q2	solve	-	Fill
Q3	0.4	3.2	Predict
Q4	3	1.8	Debug
Q5	0.0		Debug
Q6	(2, None)	(None, None)	Fill
Q7	3.0	4.33	Predict
Q8	ValueError	-1.30	Debug
Q9	≈ 0.0	$v[0] + v[1] - 2$	Debug
Q10	-np.inf	2.0	Fill